

Assignment 2: Predicting Personal Loan Acceptance using k-NN

Nusrat Nasir Nimnee

2025-09-28

Data Loading and Exploration

```
data <- read.csv("UniversalBank.csv")
str(data)
```

```
## 'data.frame':    5000 obs. of  14 variables:
## $ ID              : int  1 2 3 4 5 6 7 8 9 10 ...
## $ Age              : int  25 45 39 35 35 37 53 50 35 34 ...
## $ Experience        : int  1 19 15 9 8 13 27 24 10 9 ...
## $ Income            : int  49 34 11 100 45 29 72 22 81 180 ...
## $ ZIP.Code          : int  91107 90089 94720 94112 91330 92121 91711 93943 90089 93023
## ...
## $ Family            : int  4 3 1 1 4 4 2 1 3 1 ...
## $ CCAvg             : num  1.6 1.5 1 2.7 1 0.4 1.5 0.3 0.6 8.9 ...
## $ Education         : int  1 1 1 2 2 2 2 3 2 3 ...
## $ Mortgage          : int  0 0 0 0 0 155 0 0 104 0 ...
## $ Personal.Loan     : int  0 0 0 0 0 0 0 0 0 1 ...
## $ Securities.Account: int  1 1 0 0 0 0 0 0 0 0 ...
## $ CD.Account        : int  0 0 0 0 0 0 0 0 0 0 ...
## $ Online            : int  0 0 0 0 0 1 1 0 1 0 ...
## $ CreditCard        : int  0 0 0 0 1 0 0 1 0 0 ...
```

```
head(data)
```

```
##   ID Age Experience Income ZIP.Code Family CCAvg Education Mortgage
## 1  1  25          1     49   91107      4   1.6          1         0
## 2  2  45          19     34   90089      3   1.5          1         0
## 3  3  39          15     11   94720      1   1.0          1         0
## 4  4  35           9    100   94112      1   2.7          2         0
## 5  5  35           8     45   91330      4   1.0          2         0
## 6  6  37          13     29   92121      4   0.4          2        155
##   Personal.Loan Securities.Account CD.Account Online CreditCard
## 1              0                  1          0      0          0
## 2              0                  1          0      0          0
## 3              0                  0          0      0          0
## 4              0                  0          0      0          0
## 5              0                  0          0      0          1
## 6              0                  0          0      1          0
```

```
summary(data)
```

```
##           ID           Age           Experience           Income           ZIP.Code
## Min.      : 1   Min.    :23.00   Min.     :-3.0   Min.      : 8.00   Min.     : 9307
## 1st Qu.:1251   1st Qu.:35.00   1st Qu.:10.0   1st Qu.: 39.00   1st Qu.:91911
## Median :2500   Median :45.00   Median :20.0   Median : 64.00   Median :93437
## Mean    :2500   Mean    :45.34   Mean    :20.1   Mean    : 73.77   Mean    :93153
## 3rd Qu.:3750   3rd Qu.:55.00   3rd Qu.:30.0   3rd Qu.: 98.00   3rd Qu.:94608
## Max.     :5000   Max.     :67.00   Max.     :43.0   Max.     :224.00   Max.     :96651
##           Family           CCAvg           Education           Mortgage
## Min.      :1.000   Min.     : 0.000   Min.     :1.000   Min.     : 0.0
## 1st Qu.:1.000   1st Qu.: 0.700   1st Qu.:1.000   1st Qu.: 0.0
## Median :2.000   Median : 1.500   Median :2.000   Median : 0.0
## Mean     :2.396   Mean      : 1.938   Mean     :1.881   Mean     : 56.5
## 3rd Qu.:3.000   3rd Qu.: 2.500   3rd Qu.:3.000   3rd Qu.:101.0
## Max.     :4.000   Max.      :10.000   Max.     :3.000   Max.     :635.0
## Personal.Loan   Securities.Account   CD.Account           Online
## Min.      :0.000   Min.     :0.0000   Min.     :0.0000   Min.     :0.0000
## 1st Qu.:0.000   1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:0.0000
## Median :0.000   Median :0.0000   Median :0.0000   Median :1.0000
## Mean     :0.096   Mean      :0.1044   Mean     :0.0604   Mean     :0.5968
## 3rd Qu.:0.000   3rd Qu.:0.0000   3rd Qu.:0.0000   3rd Qu.:1.0000
## Max.     :1.000   Max.      :1.0000   Max.     :1.0000   Max.     :1.0000
##           CreditCard
## Min.      :0.000
## 1st Qu.:0.000
## Median :0.000
## Mean     :0.294
## 3rd Qu.:1.000
## Max.     :1.000
```

Data Preprocessing

```
data$Personal.Loan <- as.factor(data$Personal.Loan)
data$Education <- as.factor(data$Education)
# Check for missing values
sum(is.na(data))
```

```
## [1] 0
```

```
# Remove ID and ZIP Code as per instructions
data <- data %>% select(-ID, -ZIP.Code)

# Create dummy variables for Education manually
data$Education_1 <- ifelse(data$Education == 1, 1, 0)
data$Education_2 <- ifelse(data$Education == 2, 1, 0)
data$Education_3 <- ifelse(data$Education == 3, 1, 0)

# Remove the original Education variable
data <- data %>% select(-Education)

# Verify the dummy variables
cat("Dummy variables created successfully:\n")
```

```
## Dummy variables created successfully:
```

```
cat("Education_1 :", sum(data$Education_1), "customers\n")
```

```
## Education_1 : 2096 customers
```

```
cat("Education_2 :", sum(data$Education_2), "customers\n")
```

```
## Education_2 : 1403 customers
```

```
cat("Education_3 :", sum(data$Education_3), "customers\n")
```

```
## Education_3 : 1501 customers
```

Partition Data into Training (60%) and Validation (40%)

```
set.seed(123)
trainIndex <- createDataPartition(data$Personal.Loan, p = 0.6, list = FALSE)
train <- data[trainIndex, ]
validation <- data[-trainIndex, ]
```

k-NN Classification with k=1

```
set.seed(123)
# Train k-NN with k=1
model_k1 <- train(Personal.Loan ~ ., data = train, method = "knn",
                  preProcess = c("center", "scale"),
                  tuneGrid = data.frame(k = 1))
print(model_k1)
```

```
## k-Nearest Neighbors
##
## 3000 samples
## 13 predictor
## 2 classes: '0', '1'
##
## Pre-processing: centered (13), scaled (13)
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 3000, 3000, 3000, 3000, 3000, 3000, ...
## Resampling results:
##
## Accuracy   Kappa
## 0.9514147  0.70102
##
## Tuning parameter 'k' was held constant at a value of 1
```

Classify the Specific Customer with k=1

```
# Customer data with dummy variables as specified in assignment:
# Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2,
# Education_1 = 0, Education_2 = 1, Education_3 = 0, Mortgage = 0,
# Securities Account = 0, CD Account = 0, Online = 1, Credit Card = 1
customer <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Education_1 = 0,
  Education_2 = 1,
  Education_3 = 0,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1
)

# Predict
pred_customer_k1 <- predict(model_k1, customer)
cat("Customer classification with k=1:", as.character(pred_customer_k1))
```

```
## Customer classification with k=1: 0
```

```
if(pred_customer_k1 == 1) {
  cat("\nPrediction: Customer will ACCEPT the personal loan offer")
} else {
  cat("\nPrediction: Customer will REJECT the personal loan offer")
}
```

```
##
## Prediction: Customer will REJECT the personal loan offer
```

Choosing the Best k

```
set.seed(123)
# Train k-NN with k values from 1 to 20 for comprehensive evaluation
# Using cross-validation to find truly optimal k (not just training accuracy)
model_best <- train(Personal.Loan ~ ., data = train, method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = data.frame(k = 1:20),
  trControl = trainControl(method = "cv", number = 5))
print(model_best)
```

```
## k-Nearest Neighbors
##
## 3000 samples
## 13 predictor
## 2 classes: '0', '1'
##
## Pre-processing: centered (13), scaled (13)
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 2400, 2401, 2399, 2401, 2399
## Resampling results across tuning parameters:
##
## k Accuracy Kappa
## 1 0.9553277 0.7242593
## 2 0.9533338 0.7053305
## 3 0.9603282 0.7303034
## 4 0.9549943 0.6883490
## 5 0.9549927 0.6813678
## 6 0.9543243 0.6754048
## 7 0.9539921 0.6691965
## 8 0.9499910 0.6330426
## 9 0.9509882 0.6380566
## 10 0.9486549 0.6213020
## 11 0.9463199 0.5975303
## 12 0.9443210 0.5760181
## 13 0.9463227 0.5974206
## 14 0.9449904 0.5864319
## 15 0.9436538 0.5675298
## 16 0.9413226 0.5494488
## 17 0.9413226 0.5456843
## 18 0.9396576 0.5283767
## 19 0.9403232 0.5325555
## 20 0.9386582 0.5158999
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was k = 3.
```

```
best_k <- model_best$bestTune$k
cat("Best k:", best_k, "\n")
```

```
## Best k: 3
```

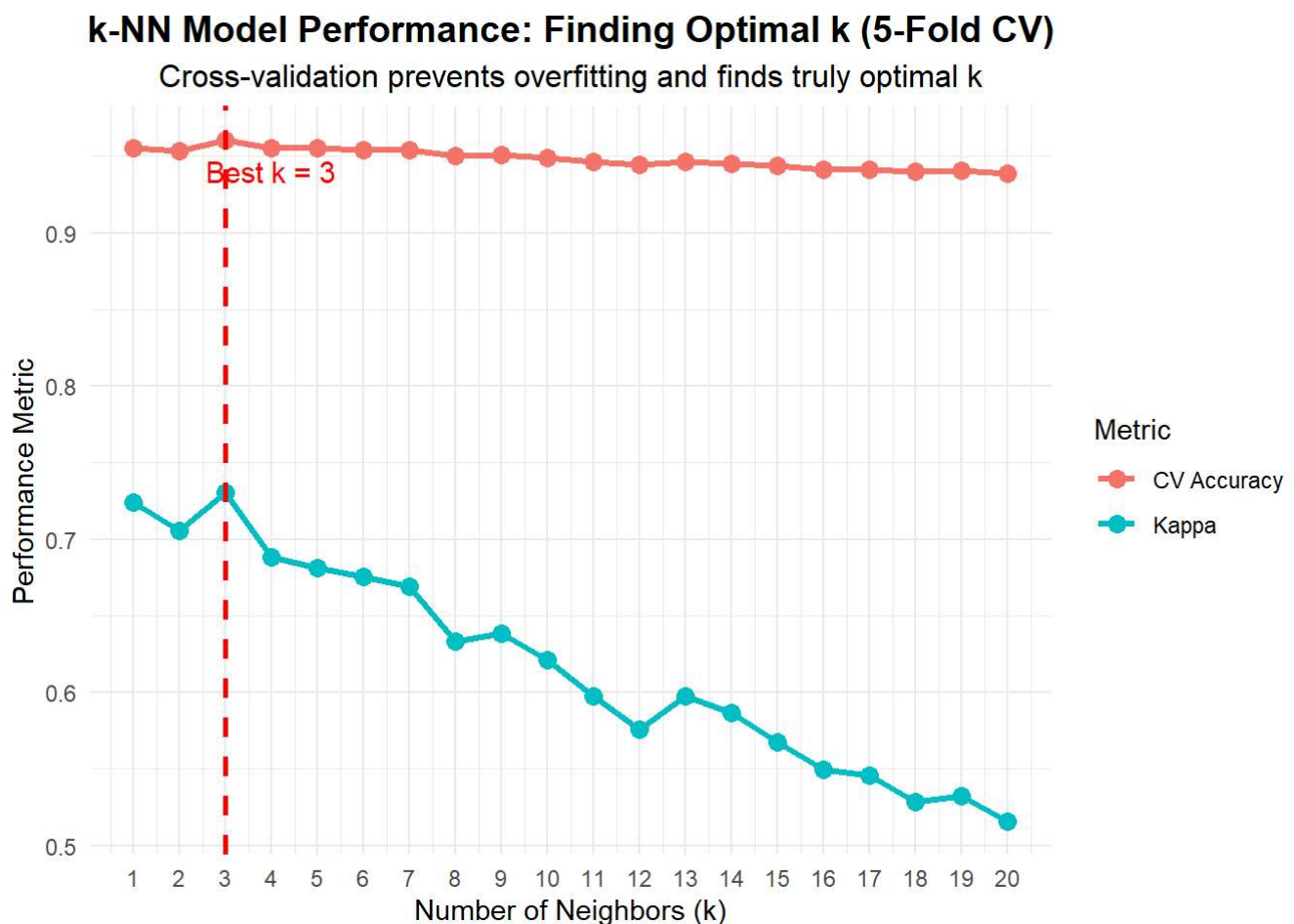
```
cat("Best cross-validated accuracy:", round(max(model_best$results$Accuracy), 4), "\n")
```

```
## Best cross-validated accuracy: 0.9603
```

```
# Create a comprehensive plot to visualize k selection
library(ggplot2)
results_df <- model_best$results

ggplot(results_df, aes(x = k)) +
  geom_line(aes(y = Accuracy, color = "CV Accuracy"), size = 1.2) +
  geom_point(aes(y = Accuracy, color = "CV Accuracy"), size = 3) +
  geom_line(aes(y = Kappa, color = "Kappa"), size = 1.2) +
  geom_point(aes(y = Kappa, color = "Kappa"), size = 3) +
  geom_vline(xintercept = best_k, linetype = "dashed", color = "red", size = 1) +
  annotate("text", x = best_k + 1, y = max(results_df$Accuracy) - 0.02,
    label = paste("Best k =", best_k), color = "red", size = 4) +
  labs(title = "k-NN Model Performance: Finding Optimal k (5-Fold CV)",
    subtitle = "Cross-validation prevents overfitting and finds truly optimal k",
    x = "Number of Neighbors (k)",
    y = "Performance Metric",
    color = "Metric") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, size = 14, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 12)) +
  scale_x_continuous(breaks = 1:20)
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## This warning is displayed once every 8 hours.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



```
# Show detailed comparison
cat("\nDetailed k comparison:\n")
```

```
##
## Detailed k comparison:
```

```
print(results_df[, c("k", "Accuracy", "Kappa")])
```

```
##      k Accuracy      Kappa
## 1     1 0.9553277 0.7242593
## 2     2 0.9533338 0.7053305
## 3     3 0.9603282 0.7303034
## 4     4 0.9549943 0.6883490
## 5     5 0.9549927 0.6813678
## 6     6 0.9543243 0.6754048
## 7     7 0.9539921 0.6691965
## 8     8 0.9499910 0.6330426
## 9     9 0.9509882 0.6380566
## 10    10 0.9486549 0.6213020
## 11    11 0.9463199 0.5975303
## 12    12 0.9443210 0.5760181
## 13    13 0.9463227 0.5974206
## 14    14 0.9449904 0.5864319
## 15    15 0.9436538 0.5675298
## 16    16 0.9413226 0.5494488
## 17    17 0.9413226 0.5456843
## 18    18 0.9396576 0.5283767
## 19    19 0.9403232 0.5325555
## 20    20 0.9386582 0.5158999
```

Confusion Matrix for Validation with Best k

```
pred_val <- predict(model_best, validation)
conf_matrix <- confusionMatrix(pred_val, validation$Personal.Loan)
print(conf_matrix)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1804   69
##           1    4  123
##
##           Accuracy : 0.9635
##           95% CI : (0.9543, 0.9713)
##    No Information Rate : 0.904
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7522
##
##    Mcnemar's Test P-Value : 6.854e-14
##
##           Sensitivity : 0.9978
##           Specificity : 0.6406
##    Pos Pred Value : 0.9632
##    Neg Pred Value : 0.9685
##           Prevalence : 0.9040
##    Detection Rate : 0.9020
##    Detection Prevalence : 0.9365
##    Balanced Accuracy : 0.8192
##
##           'Positive' Class : 0
##
```

Classify the Customer with Best k

```
# Same customer data with optimal k
pred_customer_best <- predict(model_best, customer)
cat("Customer classification with best k (k =", best_k, "):", as.character(pred_customer_best))
```

```
## Customer classification with best k (k = 3 ): 0
```

```
if(pred_customer_best == 1) {
  cat("\nPrediction: Customer will ACCEPT the personal loan offer")
} else {
  cat("\nPrediction: Customer will REJECT the personal loan offer")
}
```

```
##
## Prediction: Customer will REJECT the personal loan offer
```

Repartition into Training (50%), Validation (30%),

Test (20%)

```
set.seed(123)
trainIndex2 <- createDataPartition(data$Personal.Loan, p = 0.5, list = FALSE)
train2 <- data[trainIndex2, ]
remaining <- data[-trainIndex2, ]
valIndex <- createDataPartition(remaining$Personal.Loan, p = 0.6, list = FALSE) # 30% of total
validation2 <- remaining[valIndex, ]
test <- remaining[-valIndex, ]
```

Apply k-NN with Best k on New Partition

```
set.seed(123)
model_new <- train(Personal.Loan ~ ., data = train2, method = "knn",
  preProcess = c("center", "scale"),
  tuneGrid = data.frame(k = best_k))

# Predictions
pred_train <- predict(model_new, train2)
pred_val2 <- predict(model_new, validation2)
pred_test <- predict(model_new, test)

# Confusion Matrices
conf_train <- confusionMatrix(pred_train, train2$Personal.Loan)
conf_val <- confusionMatrix(pred_val2, validation2$Personal.Loan)
conf_test <- confusionMatrix(pred_test, test$Personal.Loan)

print("Training Confusion Matrix:")
```

```
## [1] "Training Confusion Matrix:"
```

```
print(conf_train)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 2258   61
##           1    2  179
##
##           Accuracy : 0.9748
##           95% CI : (0.9679, 0.9806)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.8369
##
##  McNemar's Test P-Value : 2.725e-13
##
##           Sensitivity : 0.9991
##           Specificity : 0.7458
##       Pos Pred Value : 0.9737
##       Neg Pred Value : 0.9890
##           Prevalence : 0.9040
##       Detection Rate : 0.9032
##   Detection Prevalence : 0.9276
##       Balanced Accuracy : 0.8725
##
##       'Positive' Class : 0
##
```

```
print("Validation Confusion Matrix:")
```

```
## [1] "Validation Confusion Matrix:"
```

```
print(conf_val)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1349   44
##           1    7  100
##
##           Accuracy : 0.966
##           95% CI : (0.9555, 0.9746)
##    No Information Rate : 0.904
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7787
##
##    McNemar's Test P-Value : 4.631e-07
##
##           Sensitivity : 0.9948
##           Specificity : 0.6944
##           Pos Pred Value : 0.9684
##           Neg Pred Value : 0.9346
##           Prevalence : 0.9040
##           Detection Rate : 0.8993
##    Detection Prevalence : 0.9287
##           Balanced Accuracy : 0.8446
##
##           'Positive' Class : 0
##
```

```
print("Test Confusion Matrix:")
```

```
## [1] "Test Confusion Matrix:"
```

```
print(conf_test)
```

```

## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 903  33
##           1   1  63
##
##           Accuracy : 0.966
##           95% CI : (0.9528, 0.9763)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : 3.243e-14
##
##           Kappa : 0.7698
##
##  Mcnemar's Test P-Value : 1.058e-07
##
##           Sensitivity : 0.9989
##           Specificity : 0.6562
##       Pos Pred Value : 0.9647
##       Neg Pred Value : 0.9844
##           Prevalence : 0.9040
##       Detection Rate : 0.9030
##       Detection Prevalence : 0.9360
##       Balanced Accuracy : 0.8276
##
##       'Positive' Class : 0
##

```

Comments on Differences

The confusion matrices show different performance across the three datasets. The training set has the highest accuracy (97.48%) because the model was trained on this data. The validation and test sets show similar accuracy (96.60% each), which is good for model generalization.

The training set shows some overfitting with higher Kappa value (0.8369) compared to validation (0.7787) and test (0.7698). However, the difference is not very large, suggesting the chosen k-value works well.

All datasets show high sensitivity (>99%) for identifying customers who will not accept loans. However, specificity is lower (65-75%), meaning the model has difficulty identifying customers who will accept loans. This happens because most customers (90.4%) do not accept loans, creating class imbalance.

The similar performance between validation and test sets indicates good model stability. The model generalizes well to new data without significant overfitting. The chosen k balances between overfitting (low k) and underfitting (high k), providing reasonable performance for loan prediction.